```
<specFlow>
  <unitTestProvider name="MsTest" />
  ...
</specFlow>
```

Alternatively, you can run the tests using NUnit, in which case you'll be running the scenarios via the NUnit test runner:

```
<specFlow>
  <unitTestProvider name="NUnit" />
  ...
</specFlow>
```

6.7.4 SpecFlow step definitions

SpecFlow step definitions are implemented as .NET methods annotated with the `[Given]`, `[When]`, or `[Then]` attributes. SpecFlow expects these methods to be public, and to be placed in a public class that's marked with the `[Binding]` attribute.

SpecFlow step definitions use regular expressions in a similar way to the Cucumber-JVM methods you saw in section 6.3. For example, one of the steps in the Flying High scenarios looks like this:

```
Given the flying distance between Sydney and Melbourne is 878 km
```

The step definition for this step might look like this:

```
[Given(@"the flying distance between (.*) and (.*) is (.*) km")]
public void GivenTheFlyingDistance(string departure,
                                   string destination,
                                   int distance)
{
   ...
}
```

❶ '@' symbol disables escape sequences

In SpecFlow, the step definition texts are typically prefixed by an `@` symbol ❶: this disables escape sequences starting with a backslash (\), making it easier to write some of the more complex Cucumber regular expressions.

A step definition method can also have several attributes, so that the same method can be used for several different variations in the scenario texts:

```
[When(@"I go from (.*) to (.*)")]
[When(@"I travel from (.*) to (.*)")]
public void GivenTheFlyingDistance(string departure, string destination)
{
    ...
}
```

If you aren't fond of regular expressions, SpecFlow also supports alternative step-definition matching rules based on the method name. You name the step definition method after the step text you want to match, using either underscores or camel case. You can also include parameter placeholders in uppercase. The parameter placeholders